

NumPy 速通指南

常用语法 + 用法 + 可直接套用的例子 | 面向强化学习 (PG agent)

0. 为什么用 NumPy

NumPy 是 Python 的数值计算库，核心是一个高性能的多维数组对象 ndarray。

核心思想：能用向量化 (vectorized) 运算，就不要写 Python 循环 —— 同样的事情，向量化往往快几十甚至上百倍。

在你的强化学习实验里，观测 obs、动作 actions、奖励 rewards、终止标志 terminals 全都是 NumPy 数组。

1. 安装与导入

```
pip install numpy
import numpy as np
```

2. 创建数组

2.1 从列表 / 常量创建

```
np.array([1, 2, 3])      # 从 list 创建一维数组
np.array([[1, 2], [3, 4]]) # 二维数组 (列表套列表)
np.zeros((3, 4))         # 全 0, 形状 (3, 4)
np.ones((2, 3))          # 全 1
np.full((2, 2), 7)        # 全 7
np.eye(3)                 # 3x3 单位矩阵
```

2.2 序列 / 随机

```
np.arange(5)             # [0 1 2 3 4]
np.arange(2, 10, 2)       # [2 4 6 8] 起始、结束(不含)、步长
np.linspace(0, 1, 5)      # 0~1 之间等距取 5 个点
np.random.randn(3, 4)     # 标准正态分布随机数
np.random.randint(0, 5, 10) # 0~4 随机整数, 共 10 个
```

小例子：做一个 3 行 4 列、全为 0.5 的数组：np.full((3, 4), 0.5)

3. 数组属性与形状操作

3.1 常用属性

```
a = np.zeros((2, 3, 4))
a.shape      # (2, 3, 4) 每个数字对应一个轴(axis)的长度
a.ndim       # 3          维数
a.dtype      # float64    数据类型
a.size       # 24         元素总数
```

3.2 改变形状与类型

```

a.astype(np.float32) # 转换成 float32 (很多库要求)
a.reshape(6, 4)      # 改变形状, 元素总数必须不变
a.reshape(-1, 4)     # -1 = 让 NumPy 自动推断那一维
a.ravel()            # 拍平成一维
a.flatten()          # 拍平成一维 (返回副本)
a.squeeze()          # 去掉长度为 1 的维度

```

`reshape(-1, ...)` 是最常用的写法: 你只想定死“每一行几个数”, 前面有几行交给它自己算。

4. 索引与切片

4.1 基础索引 (和 Python 列表类似, 但支持多维)

```

a = np.arange(12).reshape(3, 4) # [[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]]
a[0]                             # 第 1 行 -> [0 1 2 3]
a[1, 2]                          # 第 2 行第 3 列 -> 6
a[:, 1]                          # 所有行的第 2 列 -> [1 5 9]
a[1:, :2]                       # 第 2 行往后, 前 2 列

```

4.2 布尔索引 (最常用, 按条件筛数据)

```

a[a > 5]                         # 取出所有 >5 的元素, 结果是一维
a[a > 5] = 0                     # 把 >5 的元素全部改成 0
mask = a > 5                     # mask 是一个 True/False 数组, 可复用

```

4.3 花式索引 (用整数列表挑选)

```

a[[0, 2]]                       # 第 0 行和第 2 行
a[:, [0, 3]]                    # 第 0 列和第 3 列

```

5. 拼接与堆叠 (你的第一个 TODO 就用它)

```

a = np.ones((2, 3)); b = np.zeros((2, 3))
np.concatenate([a, b], axis=0)  # 沿行拼 -> (4, 3)
np.concatenate([a, b], axis=1)  # 沿列拼 -> (2, 6)
np.vstack([a, b])               # 等价于 concatenate(axis=0)
np.hstack([a, b])               # 等价于 concatenate(axis=1)
np.stack([a, b])                # 新增一个轴 -> (2, 2, 3), 别和上面搞混

```

【注意】 你的 `update` 里 `obs/actions/rewards/terminals` 是“列表套数组”, 每条轨迹一个数组。用 `np.concatenate(obs, axis=0)` 就能把所有轨迹沿第一维拼成一个大数组, 第一维变成 `batch_size`。

6. 广播 (Broadcasting)

两个数组形状不同时, NumPy 会从右往左对齐维度, 只要“该维相等, 或其中一个为 1”就能自动运算。

```

a = np.arange(12).reshape(3, 4) # (3, 4)
a + 1                             # 标量广播到每个元素
a * np.array([1, 2, 3, 4])       # (4,) 广播到每一行
a + np.array([[10], [20], [30]]) # (3, 1) 广播到每一列
a - a.mean(axis=0)               # 每列减去该列均值 (axis=0 得 (4,) 自动广播)

```

【注意】 报错 `ValueError: operands could not be broadcast` 就是形状对不上, 从右往左检查各维是否兼容。

7. 数学运算与矩阵乘法

```
a * b          # 逐元素相乘（对应位置相乘）
a @ b          # 矩阵乘法（等价 np.matmul / np.dot）
a ** 2         # 逐元素平方
np.sqrt(a)     # 逐元素开方
np.clip(a, 0, 1) # 把每个元素截断到 [0, 1] 区间
np.maximum(a, b) # 逐元素取较大者（注意不是 max）
```

8. 聚合与 axis

```
a = np.arange(12).reshape(3, 4)
a.sum()          # 全部求和 = 66
a.sum(axis=0)    # 沿 axis=0（行方向）求和 -> 每列的和（4,）
a.sum(axis=1)    # 沿 axis=1（列方向）求和 -> 每行的和（3,）
a.mean(axis=0)   # 每列均值
a.std(axis=1)    # 每行标准差
a.max()          # 最大值
a.argmax(axis=0) # 每列最大值所在的行号
a.cumsum(axis=1) # 累加和（RL 算 return 常用！）
```

理解 axis 的口诀：axis=k 就是把第 k 个轴“消掉”。

- (3,4).sum(axis=0) -> (4,), 3 没了，剩列
- (3,4).sum(axis=1) -> (3,), 4 没了，剩行

9. 高频工具函数

```
np.clip(a, 0, 1)          # 截断到 [0, 1]
np.where(cond, x, y)      # cond 为真取 x，否则取 y（向量化 if）
np.unique(a)              # 去重
np.argmax(a) / np.argmin(a) # 最大值 / 最小值的下标
np.sort(a)                # 排序
np.argsort(a)             # 排序后每个元素原来的下标
np.all(cond) / np.any(cond) # 全部为真 / 任一为真
np.transpose(a) 或 a.T    # 转置
```

np.where 小例子

```
score = np.array([90, 55, 78, 40])
result = np.where(score >= 60, '及格', '不及格')
# 输出 ['及格' '不及格' '及格' '不及格']
```

10. RL 里最常用的两个套路

10.1 用倒序累加算 discounted return

```
rewards = np.array([1.0, 2.0, 3.0, 4.0]) # 一条轨迹
gamma = 0.9
G = 0.0
```

```

returns = np.zeros_like(rewards)
for i in reversed(range(len(rewards))):
    G = rewards[i] + gamma * G      # G_t = r_t + gamma * G_{t+1}
    returns[i] = G
# returns = [7.381 7.09 6.1 4.0]

```

10.2 用 terminals 处理轨迹末端

```

# terminals[i] = 1 表示这一步是它所在轨迹的最后一步
# 需要“下一状态的价值”时，末端要清零：
next_value * (1 - terminals)  # 末端那一项自动变 0

```

你的 `_estimate_advantage` 里 GAE 的递归计算，就用 `reversed(range(batch_size))` 从后往前扫，再用 `terminals` 判断是否该截断。

11. 最常见的坑

- 形状广播对不上 —— 从右往左对齐维度检查，不匹配就 `reshape` 或指定 `axis`
- `axis=None` 是整体、指定 `axis` 是降维 —— `a.sum()` 是全和，`a.sum(axis=0)` 保留列
- `(3,)` 不是 `(3,1)` —— 一维向量和列向量广播行为不同，必要时 `.reshape(-1,1)`
- `a * b` 是逐元素乘，矩阵乘法用 `a @ b` 或 `np.dot`
- 切片是视图 —— `b = a[:2]` 改 `b` 会动 `a`；要副本用 `.copy()`
- 整数除法 —— NumPy 数组里 `/` 是浮点除，Python 列表里不是

12. 速记卡片

```

创建： array / zeros / ones / arange / linspace / random
形状： shape / reshape(-1,...) / ravel / squeeze / astype
索引： a[i, j] / a[:, 1] / a[a>5] / a[[0, 2]]
拼接： concatenate(axis=0) / vstack / hstack / stack
运算： 广播自动 / @ 矩阵乘 / clip / where
聚合： sum(axis=0) / mean / std / argmax / cumsum

```